

Ödev 20: Kendiniz Kurun

Sıfırdan gerçek bir iki paketli proje kurun, paketleyin, sonra üç başlangıç hatasını yeniden üretin ki her birini bilerek görmüş olun.

Her şey, program çıkmadan önce silinen geçici bir dizinde gerçekleşmeli.

Birinci Bölüm: İki Paket

`ProjectBuilder.java` dosyasını klasik biçimde oluşturun.

Şu düzene sahip küçük bir projeyi **yazsın, derlesin ve çalıştırsın**:

```
src/com/example/model/Item.java
src/com/example/app/Inventory.java
```

`Item`, ad ve miktar içeren bir record. Şunlara sahip olmalı:

- Diğer paketlerin çağırdığı bir **public** metot
- Public metodun kullandığı bir **paket-özel** yardımcı

`Inventory` sınıfı `Item` tipini içe aktarır, birkaç tane kurar ve yazdırır.

`ToolProvider` üzerinden `javac` ile derleyin, üretilen sınıf dosyalarını listeleyin, sonra çıktı dizinine yönlendirilmiş bir `URLClassLoader` ile `Inventory` sınıfını yükleyip çalıştırın.

`Inventory` içine, paket-özel yardımcıyı çağırmaya kalkarsa **derlenmeyecek** satırı gerçek hata metniyle birlikte gösteren bir yorum ekleyin. Kontrol edin.

Açıklayın

Dizin yolunun paket ifadesini neden yansıtmak zorunda olduğunu bir yorumda yazın. `javac` metodunun onu ne için, JVM'in ne için kullandığını söyleyin. Aynı sebep değil.

İkinci Bölüm: Paketleyin

`--main-class` ayarlanmış bir jar kurun, sonra içeriğini listeleyin.

Ayrıca ona `jar --describe-module` çalıştırın ve geleni yazdırın.

Açıklayın

İki yorum:

1. `--main-class` **gerçekte ne yaptı.** İçine yazdığı dosyayı isimlendirin ve o olmadan neyin hâlâ çalışacağını söyleyin.
2. `--describe-module` **ne bildirdi.** Bir `module-info.java` yazmadınız, peki araç neyi, nereden türetti, ve bu neden gerçek modülerlik değil?

Üçüncü Bölüm: Üç Başlangıç Hatası

Her birini yeniden üretin, yakaladığınızı yazdırın ve açıklayın:

1. **Hiç var olmamış bir sınıf için** `ClassNotFoundException`. Derleyicinin sizi neden uyaramayacağını söyleyin ve buna yol açan iki gerçek durumu isimlendirin.

2. **Var olan ama yükleyicinin yolunda olmayan bir sınıf için** `ClassNotFoundException`. Boş bir `URLClassLoader` kurun ve ondan az önce derlediğiniz bir sınıfı isteyin.

Sonra ilginç yorumu yazın: belirtilen birinci hatayla birebir aynı ve sebep tamamen farklı. Bunun gerçek bir dağıtımda neye karşılık geldiğini söyleyin.

3. **Başarısız bir statik ilklendiriciden** `NoClassDefFoundError`. Sınıfa iki kez dokunun ve her iki sonucu da yazdırın.

Doğru anlaşılmaya değer olan bu. Şunları kapsayan bir yorum yazın:

- İlk denemenin ne bildirdiği ve gerçek sebebin nerede saklandığı
- İkinci denemenin neden daha az kullanışlı bir şey bildirdiği
- **Teşhis kuralı.** Orada olduğundan emin olduğunuz bir sınıf için `NoClassDefFoundError` gördüğünüzde ne yapmalısınız?

Dördüncü Bölüm: Pratikte Modüller

Etiketlerle birlikte şunları yazdırın:

- `String` sınıfının sınıf yükleyicisi ve oradaki `null` değerinin ne anlama geldiği
- Kendi sınıfınızın yükleyicisi
- `String` sınıfının modül adı
- Kendi modülünüz ve adlandırılmış olup olmadığı

Açıklayın

Şunlar hakkında bir yorum:

- **Adsız modülün** ne olduğu, neyi okuduğu ve neyi dışa açtığı
- `--add-opens` **neden var.** Ayrıntılı olun: `java.base` neyi yapmıyor, bir kütüphane bunun sonucunda hangi istisnayı alıyor, ve bayrak neyi değiştiriyor?

Kabul Kriterleri

- ☐ `java ProjectBuilder.java` ile çalışıyor ve geçici dizinini siliyor
- ☐ İki paket yazılıyor, derleniyor ve çalıştırılıyor
- ☐ `Item` hem public hem paket-özel bir üyeye sahip
- ☐ Derlenmeyen çağrı gerçek hata metniyle belgelenmiş
- ☐ Bir yorum izin kuralını hem `javac` hem JVM tarafından açıklıyor
- ☐ `--main-class` ile bir jar kuruluyor ve içeriği listeleniyor
- ☐ `--describe-module` çıktısı yazdırılıyor ve açıklanıyor
- ☐ Üç başarısızlık da gerçek mesajlarıyla yeniden üretiliyor
- ☐ Birinci ve ikinci başarısızlıklar belirtile değil sebeple ayırt ediliyor
- ☐ Statik ilklendirici durumu her iki denemeyi de gösteriyor
- ☐ `NoClassDefFoundError` için bir teşhis kuralı belirtiliyor
- ☐ Adsız modül ve `--add-opens` ikisi de açıklanıyor

İleri Seviye

Yalnızca `com.example.model` paketini dışa açan, `com.example.app` paketini **açmayan** gerçek bir `module-info.java` ekleyin. Modül olarak derleyin ve `--module-path` ile çalıştırın.

Sonra dışarıdan, dışa açılmamış pakete yansıma yapmayı deneyin ve ne aldığınızı kaydedin.

Güçlü kapsüllemenin ikinci bölümdeki jar'ın vermediği neyi verdiğini ve karşılığında neye mal olduğunu bir yorumda yazın.

İpucu, ikinci başarısızlık başarısız olmuyorsa

Sıradan bir `new URLClassLoader(urls)` önce ebeveynine devreder, ve ebeveyn muhtemelen sınıfınızı bulabilir. Devretmeyi durdurmak için ebeveyn olarak `null` geçirin, o zaman yükleyicinin gerçekten bakacak hiçbir yeri kalmaz.